For: ATHE Level 5 Extended Diploma in Computing

Unit 9 Advanced Programming Assignment

Assignment Scenario

You are working as an Assistant Software Developer in ModernDayTech, an agile start-up which subcontracts the creation of software engineering solutions from other organisations within the digital transformation sector.

Having successfully completed your internship with the company, senior members of the team have identified you as a potential “buddy”/learning peer for new recruits.

One technical area where it is felt that potential recruits are typically weaker is in the understanding of the Object-Oriented Programming paradigm. As a result, you have been tasked to create several support documents, code samples and projects to illustrate its best practice.

Task 1

Produce a document for peers which provides both a theoretical and practical introduction to the OOP paradigm and SOLID principles:

- Explain the aim of the 5 key SOLID principles and the many ways code can be evaluated to ensure compliance with design patterns.
- Define the four pillars of Object-Oriented Programming with a coded example for each.

Remember to target your content for a mixed but educated audience. You should make sure that the content is varied and interesting, utilising well-chosen, real-world analogies and learning techniques to impart the technical information successfully.

Extension activities:

To achieve a **Merit**, your manager has asked you to create supplementary material in the form of a sample complex application – of your choosing – which uses all four pillars of OOP.

To achieve a **Distinction**, make sure that your chosen complex application also has two or more SOLID principles applied.

Learning outcomes and assessment criteria:

LO1 AC 1.1, 1.2, 1M1, 1D1

Task 2

Due to emerging concerns regarding AppSec (application security), your organisation is leaning heavily into DevSecOps as a framework for robust solution design and implementation. Unfortunately, it is a framework with which few new recruits are actively familiar.

Produce a document for peers which provides a suitable introduction to DevSecOps, ensuring that you:

- Explain the importance of implementing Shift Left as a DevSecOps best practice.
- Explain the problems DevSecOps aims to solve during the development lifecycle.

Remember to target your content for a mixed but educated audience. Once again, you should make sure that the content is varied and interesting, utilising well-chosen real-world analogies and learning techniques to impart the technical information successfully.

Extension activities:

To achieve a **Merit**, your manager has asked you to extend your DevSecOps coverage:

- Design suitable SCA, SAST and DAST stages using appropriate tools and configuration.

To achieve a **Distinction**, you must also:

- Implement a DevSecOps CI/CD pipeline using an open-source automation server.

Learning outcomes and assessment criteria:

LO2 AC 2.1, 2.2, 2M1, 2D1

Task 3

Senior stakeholders have accepted your argument that a “model” application is required which is typical of the workloads tackled by your teams. One such concept is the use of large dataset applications, as might be tackled for machine learning.

A sample of non-commercial datasets can be found here:

https://www.datasetlist.com/

Select a suitable dataset from this resource (or similar) and:

- Demonstrate designing and building a large dataset application to a client's requirement.
- Describe examples of SOLID principles and clean coding techniques applied as part of the solution.

Note: in this example, the client’s requirement is your choice, but must be suitably complex so as to fulfil all the assessment criteria provided.

Learning outcomes and assessment criteria:

LO3 AC 3.1, 3.2

Task 4

Once the large dataset application has been written to the client specification, it is necessary to test it. Since manual testing is too laborious, you must:

- Build an automated test to a set specification.
- Explain the benefits of automated testing over manual testing.

Note: the set specification is once again your choice, but you must be mindful of the different types of security tests available, i.e. SCA, SAST, DAST etc. and ensure that these are fully represented and applied.

Extension activities:

To achieve a **Merit**, make sure that you can evidence that you have been able to:

- Demonstrate the utilisation of automated testing tools

Note: this automated test would logically utilise the automated test stage previously designed and built.

Learning outcomes and assessment criteria:

LO3 AC 3.3, 3.4, 3M1

Guidelines for assessors

The assignments submitted by learners must achieve the learning outcomes and meet the standards specified by the assessment criteria for the unit. To achieve a merit or distinction grade, the learners must demonstrate that they have achieved all the criteria set for these grades. Where work for the pass standard is marginal, assessors can take into account any extension work completed by the learners. The suggested evidence listed below is how learners can demonstrate that they have met the required standards. The command verbs contained within the AC are highlighted in bold in the suggested evidence boxes.

Task number	LOs and AC	Suggested evidence PASS	Suggested additional evidence MERIT	Suggested additional evidence DISTINCTION
1.	LO1 AC 1.1, 1.2,1M1, 1D1	Task 1 focuses on the production of varied learning material that helps learners to understand the importance of implementing SOLID principles on the OOP paradigm. **LO1 AC 1.1:** Learners should be able to accurately **explain** the five key SOLID principles (SRP, open-closed, Liskov substitution, interface segregation, and dependency inversion). They must also be able to **explain** the many ways code can be evaluated to ensure compliance with design patterns, e.g. through code review, testing etc. **LO1 AC 1.2:** Learners must accurately **define** the four pillars of OOP, i.e. abstraction, encapsulation, inheritance, and polymorphism etc. This is best achieved via the use of a theoretical example, e.g. bank account classes, which can demonstrate all these aspects.	**LO1 1M1:** Learners must **design and implement** a coded solution for a complex problem which uses all four pillars of OOP, i.e. abstraction, encapsulation, inheritance, and polymorphism etc. Learners can evidence this through code which creates classes (e.g. for encapsulation), creates derived classes (e.g. for inheritance) etc.	**LO1 1D1:** Learners should be able to include the use of two or more SOLID principles in their coded solution for a complex problem (of their choice). For example, this might include the use of classes or methods which clearly demonstrate SRP, Liskov substitution principle etc. An example of the latter might be passing super- and derived-class objects into a function and calling the same method but getting different outputs.

2.	LO2 AC 2.1, 2.2, 2M1, 2D1	Task 2 focuses on the learner being able to understand the importance DevSecOps as a set of practices, cultural approaches, and tools in software development. **LO2 AC 2.1:** This would be **explained** through a comparison of DevOps and DevSecOps pipelines with the security aspects being shifted earlier (i.e. to the left) before costly problems/bugs are "baked" into the solution. **LO2 AC 2.2:** This would be evidenced through the creation of a document which **explains** the various problems that DevSecOps aims to solve, i.e. reducing the cost of fixing problems in a solution by identifying them as early as possible in the development pipeline rather than at the end (as with traditional testing models).	**LO2 2M1:** Learners should be able to able to **design** a document/process which details appropriate SCA, SAST and DAST test stages, along with the correct tools and configuration required to perform them.	**LO2 2D1:** Learners should be able to able to **implement** a DevSecOps CI/CD pipeline using a tool such as Jenkins or GitLab Actions.
3.	LO3 AC 3.1, 3.2	Task 3 focuses on the learner designing and building a large dataset application using SOLID principles and clean coding techniques. **LO3 AC 3.1:** This would be evidenced by the learner selecting a large dataset (from recommended sources) and then processing it to achieve a set objective. Typically, this may be a machine learning related task. **LO3 AC 3.2:** This would be evidenced by the learner **describing** examples of SOLID		

		principles and clean coding techniques present within the solution they have created. This may be achieved through targeted commenting.		
4.	LO3 AC 3.3, 3.4, 3M1	Task 4 focuses on applying automated techniques to test a previously created large dataset application. **LO3 AC 3.3:** Learners should be able to **build** an automated test (including elements of SCA, SAST, DAST etc) to a set specification of their choosing. This can be evidenced through screen captures, videos, annotated descriptions etc. **LO3 AC 3.4:** Learners should provide a short section of accompanying documentation for their automated test which **explains** its advantages over traditional testing methods, particularly with reference to a DevSecOps process and a suitably staged CI/CD pipeline. Learners may draw upon aspects of previous automated tests created to illustrate their points.	**LO3 3M1:** Learners should be able to able to **demonstrate** the utilisation of an automated test within a CI/CD. Ideally, this should demonstrate both success and failure (for contrast). This can be evidenced through screen captures, videos, annotated descriptions etc.	